

Distributed Databases using Replication

This assignment cuts to heart of distributed systems – how to work with the CAP theorem to safely manage shared state in a distributed system! Feel free to discuss this with anyone you like, but please submit your own individual work. Identify any and all elements of your assignment that was assisted by AI, and please be sure you understand all of the work you submit.

Key-Value (KV) Service

You will write an in-memory Key-Value store as the foundation for a Leader-Follower distributed database and a Leaderless distributed database. Both these databases are essentially hashtable-as-a-service; a simplified version of redis. The API for both databases is identical. The API has two very simple endpoints:

1. `set(Key:string, Value: string)` stores the Value under that key, and returns 201-Created. The key cannot be empty. The empty string is a valid value. This endpoint is also known as `write`.
2. `get(Key:String)` returns the value (and 200-OK) if the Key is present, otherwise return 404. This is also known as `read`.

For simplicity the KV service does NOT need to persist any data to the file system; all values are stored in memory and are lost when the service shuts down.

You will need to create Dockerfiles and deploy them just like you did for the previous assignments.

The two implementations of the KV service should add delays to simulate the real-world time delays that would occur when writing to storage, and make it easier to test the inconsistency window. If everything commits really quickly then the race condition windows are very small, and it is hard to create conflicts. Specifically, when the Leader is updating its Followers during a `set` operation, the Leader should sleep for 200ms following each message to a Follower. The Follower should also sleep for 100ms when it receives the update before responding.

When the Leader receives a `get` operation it does not need to sleep, but any Follower that receives a read request from the Leader should sleep for 50ms before responding.

Leader-Follower Database

Add the necessary endpoints (and the code that implements them) so that the cluster can be easily configured to for N , R , and W as defined in class Nov 5. You should have five nodes in the set of nodes that make up the distributed database, i.e. $N = 5$. That is one Leader and four Followers. All write requests go to the Leader, which replicates the data to all other nodes. Therefore, your load-test client needs to know the IP address of the Leader and only send writes to it. Reads can go to any instance.

Replication Strategies

Implement the following replication strategies:

1. $W=5$ and $R=1$, meaning that on a write every node must be updated before the Leader responds to the client. A read only needs to read the value on the leader node.

2. $W=1$ and $R=5$, meaning that only the Leader needs to be updated before responding for a write request. However, a read must fetch the data from every node and return the most recent version.
3. Use a quorum of $R=3$, $W=3$. Reads must return the most recent version of the data.

You will need logical version numbers on the KV pairs for these algorithms.

Leaderless Database

This is the same situation but there is no distinguished leader. Read and Write requests can go to any node. Configure the case where $W=N$ and $R=1$.

When a node receives a write request it becomes the **Write Coordinator** for that write request. It must coordinate writes to every other node, wait for them to complete, and only then return 201-Created to the client.

When a node receives a read request from the client it returns its own value. All writes must be propagated by the node that is the Write Coordinator to all other nodes. It is possible for a read to arrive at an un-updated node, and therefore there is an inconsistency window. This flaw is acceptable because the purpose of the following section is to show it happening in practise.

Testing Consistency

Write a unit that tests the Leader database for consistency:

1. The unit test send `set` to the Leader Node.
2. After the Leader acknowledges the write, read from the Leader. That should return consistent data.
3. After the Leader acknowledges the write, read from a Follower. That should return consistent data.

To sneakily test the internal workings of the Leader database, add an endpoint `local_read` which just returns the KV value on that node. During a set operation that endpoint will often return inconsistent data. Adding sneaky (API-breaking) read methods like `local_read` is a common testing tactic, often that is only used during testing.

1. Send `set` to the Leader.
2. Within the update window the test send `local_read` to the other Followers. This might show inconsistency. If you repeat this often at high load it should happen.

For the Leaderless database, write a unit test that exposes the inconsistency window:

1. The unit test writes a key-value pair to a random node. That node is now the Write Coordinator.
2. Within the update time window test read (i.e. send `get` messages) to other nodes. These returns should be inconsistent.
3. After the Coordinator acknowledges the write, read from the Coordinator. That should be consistent.
4. After the Coordinator acknowledges the write, read from another node. That should be consistent.

Load Testing

Create a load-test client to explore the read-write performance of the various databases. In order to produce data for stale reads and trigger the "return the most recent value" logic, you must ensure that your load test generator produces data that is "local-in-time", i.e. reads and writes occur to the same key clustered closely

together in time. Some suggestions: you can either simply use a smaller number of keys (easy) or a more complex algorithm that clusters reads and writes to the same key closely together in time, but still has a large number of keys.

Perform load-test runs with the following read-write ratios on the three Leader and one Leaderless configuration and record your results:

Writes	Reads
1%	99%
10%	90%
50%	50%
90%	10%

For each run record the:

1. Latency for each request, and
2. Number of stale reads from Followers or non-Write Coordinator nodes. You will need to track the version of the data in your client as well to detect staleness.

Create graphs of your results that show the:

1. Distribution of latency for reads and separately for writes. These graphs should show any "long tail," if it exists.
2. Distribution of the time intervals between reading and writing the same key, as generated by your load-tester.

Discuss your results. How does your test generator work? How does it guarantee that reads and writes frequently occur on the same Key?

Submission Requirements

1. All Code and configurations to be in a Khoury git repository.
 1. URL of the git repo.
 2. Code for the Leader-Follower database
 3. Code for the Leaderless database
 4. Code for the load tester
 5. Dockerfiles
 6. Unit tests that "prove" that it works
 7. Any other configuration
2. Explain how your code works!
 1. Explain how the different N/R/W cases are implemented in your code: What happens when a write message arrives? what happens when a read arrives? Don't just describe the code, talk about how the whole thing works, which bits are tricky, how you handle errors etc. **Imagine you are handing this code over to a friend to look after – what would you tell them so that they have an easier time.**
 2. Show that you understand what all AI-generated code does.
 3. All members of the team need to be able to do this!

3. A PDF report with

1. Graphs of Time Latencies, for each of the four read-write ratios:

1. Reads
2. Writes
3. Time interval between reads and writes of the same key.

2. Discussion of those results

1. Which type of Leader-Follower does best with each read/write ratio?
2. Do not just *describe* the graphs in English, we are looking for reasoned analysis as to *why* you got those results.
3. Which kind of database would be best used for what kind of application?